FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Elijah Paul M. Nonan
Section:	AN-43
Professor:	Ms. Crisola Tan
Date:	11/04/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description

- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.
Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

Source Code

```
"""
TA3 Case-Based Reasoning: Intelligent Tutoring System.

The program retrieves a similar learning case, reuses its strategy, adapts the
feedback, and retains the new case in a local JSON file.
"""

import json
from pathlib import Path


DATA_FILE = Path(__file__).with_name("its_retained_cases.json")

# Each case has an ID, a short problem description, student attributes, and the
learning strategy that helped in that situation.
SAMPLE_CASES = [
    {
        "id": "ITS-001",
        "description": "A learner confuses multiplication and addition facts.",
        "attributes": {
            "subject": "math",
            "error_type": "conceptual",
            "mastery": 30,
        },
        "solution": "Use visual examples and explain the concept step by step.",
    },
    {
        "id": "ITS-002",
        "description": "A learner understands fractions but makes calculation errors.",
        "attributes": {
            "subject": "math",
            "error_type": "calculation",
            "mastery": 55,
        },
        "solution": "Model one worked example, then give short calculation practice.",
    },
    {
        "id": "ITS-003",
```

```
        "description": "A learner applies the wrong rule when solving equations.",
        "attributes": {
            "subject": "math",
            "error_type": "conceptual",
            "mastery": 60,
        },
        "solution": "Review the rule with a worked example and ask the learner to
explain each step.",
    },
    {
        "id": "ITS-004",
        "description": "A learner makes spelling errors in English writing.",
        "attributes": {
            "subject": "english",
            "error_type": "careless",
            "mastery": 70,
        },
        "solution": "Use a short editing checklist and ask the learner to review their
own work.",
    },
    {
        "id": "ITS-005",
        "description": "A learner struggles to remember science vocabulary.",
        "attributes": {
            "subject": "science",
            "error_type": "memory",
            "mastery": 40,
        },
        "solution": "Use flashcards and spaced review, with a simple example for each
term.",
    },
]


def load_case_base():
    """Load retained cases, keep the built-in sample cases available."""
    cases = list(SAMPLE_CASES)

    if not DATA_FILE.exists():
        return cases

    try:
        with DATA_FILE.open("r", encoding="utf-8") as file:
            saved_cases = json.load(file)

        if not isinstance(saved_cases, list):
            raise ValueError("The JSON root must be a list.")

        sample_ids = {case["id"] for case in SAMPLE_CASES}
        for case in saved_cases:
            if (
                isinstance(case, dict)
                and isinstance(case.get("id"), str)
                and isinstance(case.get("attributes"), dict)
                and isinstance(case.get("solution"), str)
                and case["id"] not in sample_ids
            ):
                cases.append(case)
```

```
    except (OSError, json.JSONDecodeError, ValueError) as error:
        print(f"Notice: Could not load saved cases ({error}).")
        print("The built-in sample cases will still be available.")

    return cases


def save_retained_cases(cases):
    """Save only newly retained cases, leaving built-in examples in the code."""
    sample_ids = {case["id"] for case in SAMPLE_CASES}
    retained_cases = [case for case in cases if case["id"] not in sample_ids]

    with DATA_FILE.open("w", encoding="utf-8") as file:
        json.dump(retained_cases, file, indent=2)


def ask_choice(prompt, choices):
    """Ask until the user enters one of the permitted choices."""
    allowed = ", ".join(choices)

    while True:
        answer = input(f"{prompt} ({allowed}): ").strip().lower()
        if answer in choices:
            return answer
        print(f"Please enter one of: {allowed}.")


def ask_mastery():
    """Ask for a whole-number mastery level from 0 to 100."""
    while True:
        answer = input("Mastery level (0-100): ").strip()
        try:
            mastery = int(answer)
            if 0 <= mastery <= 100:
                return mastery
        except ValueError:
            pass
        print("Enter a whole number from 0 to 100.")


def get_new_problem():
    """Collect and return the new student's problem and attributes."""
    print("\nEnter the learner's new problem.")
    subject = ask_choice("Subject", ["math", "english", "science"])
    error_type = ask_choice(
        "Main error pattern",
        ["conceptual", "calculation", "careless", "memory"],
    )
    mastery = ask_mastery()

    description = input("Brief problem description: ").strip()
    if not description:
        description = f"Learning difficulty in {subject} ({error_type} errors)."

    return {
        "description": description,
        "attributes": {
```

```
            "subject": subject,
            "error_type": error_type,
            "mastery": mastery,
        },
    }


def similarity(new_attributes, old_attributes):
    """Calculate a transparent weighted score from 0 to 100.

    Subject contributes 40 points, error pattern 35 points, and mastery 25 points.
Mastery similarity is normalized: a 0-point difference scores 1.0, while a 100-point
difference scores 0.0.
    """
    subject_score = (
        1.0 if new_attributes["subject"] == old_attributes["subject"] else 0.0
    )
    error_score = (
        1.0
        if new_attributes["error_type"] == old_attributes["error_type"]
        else 0.0
    )
    mastery_score = 1.0 - (
        abs(new_attributes["mastery"] - old_attributes["mastery"]) / 100.0
    )

    weighted_score = (
        0.40 * subject_score
        + 0.35 * error_score
        + 0.25 * mastery_score
    )
    return weighted_score * 100


def retrieve_best_case(new_attributes, cases):
    """Rank cases by similarity; break ties by case ID for stable results."""
    ranked_cases = [
        (similarity(new_attributes, case["attributes"]), case)
        for case in cases
    ]
    ranked_cases.sort(key=lambda item: (-item[0], item[1]["id"]))
    return ranked_cases


def adapt_solution(retrieved_case, new_attributes):
    """Reuse the retrieved strategy and adapt it to the new learner."""
    subject = new_attributes["subject"]
    error_type = new_attributes["error_type"]
    mastery = new_attributes["mastery"]
    base_strategy = retrieved_case["solution"]

    # Keep the retrieved case's strategy as the starting point, then tailor it.
    if error_type == "conceptual":
        adaptation = "Add a simple explanation and a visual or concrete example."
    elif error_type == "calculation":
        adaptation = "Add several short practice questions and check each step."
    elif error_type == "careless":
        adaptation = "Add a brief self-check checklist before submitting work."
```

```
    else:  # memory
        adaptation = "Add flashcards and review the material again after a short
delay."

    if mastery < 40:
        level_guidance = "Begin with one small step and provide close guidance."
    elif mastery < 70:
        level_guidance = "Use a worked example, then let the learner try a similar
task."
    else:
        level_guidance = "Give a more challenging task and ask the learner to explain
their reasoning."

    subject_guidance = {
        "math": "Use numbers or equations from the current math topic.",
        "english": "Use a short reading or writing example from the current lesson.",
        "science": "Use a familiar science example or diagram from the current topic.",
    }[subject]

    return (
        f"Start from the retrieved strategy: {base_strategy}\n"
        f"Adaptation for this learner: {adaptation}\n"
        f"Mastery adjustment: {level_guidance}\n"
        f"Subject-specific practice: {subject_guidance}"
    )


def next_case_id(cases):
    """Create the next sequential retained-case ID."""
    highest_number = 0
    for case in cases:
        case_id = case.get("id", "")
        if case_id.startswith("ITS-"):
            try:
                highest_number = max(highest_number, int(case_id[4:]))
            except ValueError:
                continue
    return f"ITS-{highest_number + 1:03d}"


def display_case_base(cases):
    """Print a concise view of every case currently in the case base."""
    print("\nUPDATED CASE BASE")
    print("-" * 72)
    for case in cases:
        attributes = case["attributes"]
        print(
            f"{case['id']} | {attributes['subject']} | "
            f"{attributes['error_type']} | mastery {attributes['mastery']}"
        )
        print(f"  Problem: {case['description']}")
        print(f"  Solution: {case['solution']}")


def process_scenario(cases):
    """Demonstrate Retrieve, Reuse, Revise, and Retain for one problem."""
    new_problem = get_new_problem()
    attributes = new_problem["attributes"]
```

```
    print("\nNEW PROBLEM")
    print("-" * 72)
    print(f"Description: {new_problem['description']}")
    print(f"Subject: {attributes['subject']}")
    print(f"Main error pattern: {attributes['error_type']}")
    print(f"Mastery: {attributes['mastery']}/100")

    # RETRIEVE: calculate and show the similarity ranking.
    ranked_cases = retrieve_best_case(attributes, cases)
    print("\nRETRIEVE: Similarity ranking")
    for score, case in ranked_cases:
        print(f"  {case['id']}: {score:.1f}%")

    best_score, best_case = ranked_cases[0]
    print(
        f"\nMost similar case: {best_case['id']} "
        f"({best_score:.1f}% similar)"
    )
    print(f"Problem: {best_case['description']}")

    # REUSE: take the solution associated with the retrieved case.
    print("\nREUSE: Retrieved case solution")
    print(best_case["solution"])

    # REVISE: adapt the retrieved strategy to the new attributes.
    final_solution = adapt_solution(best_case, attributes)
    print("\nREVISE: Adapted learning recommendation")
    print(final_solution)

    # RETAIN: store the solved problem and adapted recommendation.
    new_case = {
        "id": next_case_id(cases),
        "description": new_problem["description"],
        "attributes": attributes,
        "solution": final_solution.replace("\n", " "),
    }
    cases.append(new_case)

    try:
        save_retained_cases(cases)
        print(f"\nRETAIN: Saved new case {new_case['id']} to {DATA_FILE.name}.")
    except OSError as error:
        print(f"\nRETAIN: Could not save to JSON file ({error}).")
        print("The new case remains in the case base for this session.")

    display_case_base(cases)


def main():
    """Run the ITS application until the user chooses to stop."""
    cases = load_case_base()
    print("=" * 72)
    print("CASE-BASED REASONING: INTELLIGENT TUTORING SYSTEM")
    print("=" * 72)
    print(f"Loaded {len(cases)} case(s), including five built-in examples.")

    while True:
```

```
        process_scenario(cases)
        again = input("\nProcess another learner? (y/n): ").strip().lower()
        if again not in ("y", "yes"):
            print("\nSession complete.")
            break


if __name__ == "__main__":
    main()
```

Documentation

Title: Intelligent Tutoring System using Case-Based Reasoning

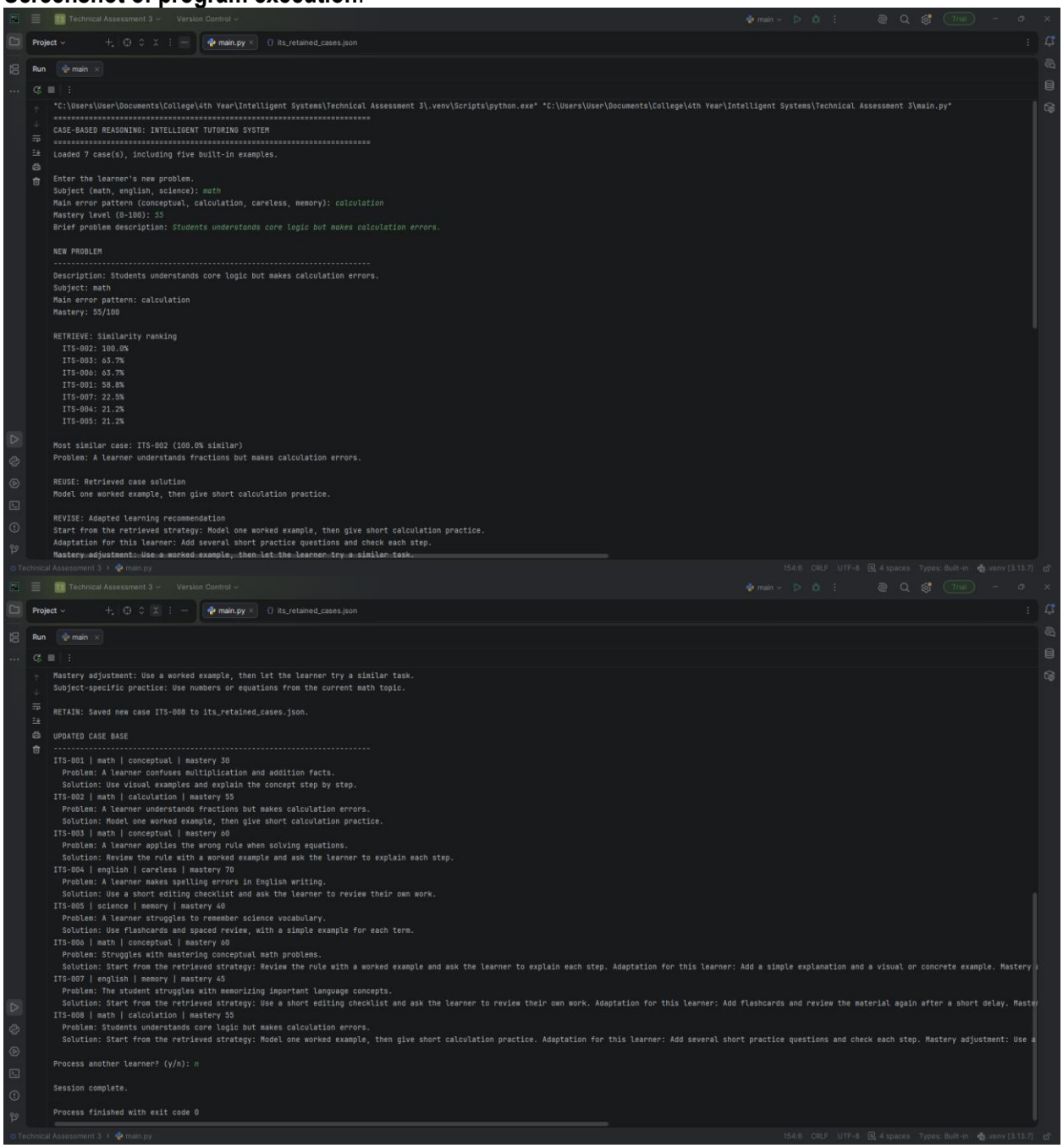
Description: This console application demonstrates how a case-based reasoning system can recommend learning support for a student by comparing the student's difficulty with previously solved learning cases. The program performs Retrieve, Reuse, Revise, and Retain for each scenario.

Case base and attributes: Five example cases are included in the source code. Each case has a unique ID, a problem description, a solution, and three attributes: subject (math, english, or science), main error pattern (conceptual, calculation, careless, or memory), and mastery level from 0 to 100. Newly solved cases are saved in its_retained_cases.json beside the Python file and loaded in later runs. If that file is missing or malformed, the built-in cases remain available.

Similarity assessment: The program uses weighted attribute scoring. Matching subject contributes 40% of the score; matching error pattern contributes 35%. Mastery contributes 25%, with its score normalized by the distance between two mastery levels: identical levels receive full credit, while a difference of 100 points receives none. The program displays each score as a percentage. Ties are listed in case-ID order for consistent output.

Adaptation Logic: The retrieved case's learning strategy is the starting point for the recommendation. The system then adjusts the feedback to the new student's error pattern, mastery level, and subject. For example, a learner with low mastery receives close guidance; a learner making careless errors gets a self-check checklist. This lets the retrieved solution influence the result while tailoring it to the new scenario.

Screenshot of program execution:



Reflection

Developing the intelligent system for my Technical Assessment was a heavy test of my coding fundamentals. The main problem I encountered in accomplishing this task was simply understanding what Case-Based Reasoning (CBR) actually is and implementing it in the right way without relying on pre-built shortcuts. However, once I pushed through that initial roadblock, my focus entirely shifted to how the core logic operates. I learned that CBR is not just some complex AI magic, instead, it is a highly structured way for a program to learn directly from past experiences, how it mimics human problem-solving: the system retrieves the closest historical case, reuses the old strategy, revises it to fit the current student's specific problem, and then retains that new solution to make the database smarter for the future.